Dominio 3: configurazione e workflow di Claude Code (20%)

Questo dominio tratta la gerarchia e il caricamento di CLAUDE.md, i comandi personalizzati e le skill, le regole specifiche per percorso, la modalità plan rispetto all'esecuzione diretta, il perfezionamento iterativo e l'integrazione CI/CD.

3.0 Ambienti: dove viene eseguito Claude Code

Claude Code non è più solo una CLI. Ogni ambiente si connette allo **stesso motore**, quindi file CLAUDE.md, impostazioni, skill e server MCP funzionano allo stesso modo ovunque. Il punto importante dal punto di vista architetturale è che la configurazione è portabile, così come la sessione.

Ambiente	Descrizione	Installazione / punto d'accesso
Terminale	CLI completa	`curl -fsSL https://claude.ai/install.sh | bash`, poi `claude`
VS Code / Cursor	Estensione con differenze inline, menzioni con @ e revisione dei piani	Marketplace delle estensioni
JetBrains	Plugin per IntelliJ, PyCharm e WebStorm (richiede la CLI)	JetBrains Marketplace
App desktop	Applicazione autonoma: differenze visive, sessioni parallele, attività pianificate e sessioni cloud	Download per macOS / Windows
Web	Sessioni nel browser senza configurazione locale; attività prolungate su repository non presenti in locale	claude.ai/code
Dispositivo mobile	App iOS / Android per le stesse sessioni cloud	App Claude

Portabilità delle sessioni

Obiettivo	Meccanismo
Iniziare in locale e terminare nel cloud	`claude --cloud "<task>"` (`--remote` è un alias deprecato)
Trasferire una sessione Web/cloud nel terminale	`claude --teleport`
Eseguire un'attività come sessione locale in background	`claude --bg "<task>"`, poi `claude agents` / `claude attach <id>` / `logs` / `stop` / `respawn` / `rm`
Copiare la sessione corrente in una sessione in background	`/fork [prompt]` (per un *subagente* fork che restituisce il risultato usa `/subtask`; `/branch` passa a una copia)
Passare la sessione corrente del terminale all'app desktop	`/desktop`
Controllare da un telefono o da un altro browser una sessione locale in esecuzione, oppure trasferirla dall'app Claude a una sessione locale in background (v2.1.273+)	Remote Control
Inviare eventi esterni (Telegram, Discord, iMessage, webhook) a una sessione	Channels
Instradare attività dalla chat del team	`@Claude` in Slack

Pianificazione

Esistono tre meccanismi distinti. Per l'esame, la differenza importante è *dove vengono eseguiti*:

Meccanismo	Dove viene eseguito	Uso
Routines (`/schedule`, Web o desktop)	Nel cloud: continuano a funzionare anche quando il computer è spento; possono attivarsi anche in seguito a chiamate API o eventi GitHub	Revisioni PR mattutine, analisi notturne degli errori CI, audit settimanali delle dipendenze
Attività pianificate desktop	Sul computer, con accesso ai file locali e agli strumenti	Tutto ciò che richiede il file system locale o una rete privata
`/loop`	Nella sessione CLI corrente	Controlli periodici rapidi durante una sessione; ometti l'intervallo e Claude regolerà autonomamente il ritmo tra le iterazioni

---

3.1 Gerarchia di CLAUDE.md

Che cos'è CLAUDE.md?

I file CLAUDE.md contengono istruzioni che indicano a Claude Code come comportarsi in un determinato contesto. Sono simili a `.editorconfig` o `.eslintrc`, ma per un agente IA: definiscono standard di programmazione, convenzioni del progetto, restrizioni sugli strumenti e regole dei workflow.

Ordine di caricamento (dal più generale al più specifico)

Claude Code carica i file CLAUDE.md secondo una gerarchia. I file più specifici prevalgono su quelli più generali:

```
1. Policy gestita (organizzazione intera, controllata dall'amministratore)
   └── /etc/claude-code/CLAUDE.md                          (Linux, WSL)
   └── /Library/Application Support/ClaudeCode/CLAUDE.md    (macOS)
   └── C:\Program Files\ClaudeCode\CLAUDE.md                (Windows)

2. Livello utente (personale, NON soggetto al controllo versione)
   └── ~/.claude/CLAUDE.md

3. Livello progetto (condiviso tramite controllo versione)
   └── ./CLAUDE.md  oppure  ./.claude/CLAUDE.md

4. Progetto locale (personale, escluso da Git)
   └── ./CLAUDE.local.md

5. Livello directory (caricato su richiesta)
   └── ./src/api/CLAUDE.md (caricato quando Claude legge file in src/api/)
   └── ./tests/CLAUDE.md (caricato quando Claude legge file in tests/)
```

Distinzione fondamentale: CLAUDE.md a livello di progetto viene incluso nel commit Git e condiviso con il team. Quello a livello utente è personale e locale. `CLAUDE.local.md` è personale per il singolo progetto (inseriscilo in `.gitignore`). Quello a livello di directory viene caricato dinamicamente quando Claude lavora in quella directory.

Interoperabilità con AGENTS.md

Dalla versione v2.1.277, se un progetto **non** contiene `CLAUDE.md`, Claude Code legge `AGENTS.md` come alternativa (su tutti i provider dalla versione v2.1.281; l'opzione si trova in «Istruzioni del progetto» in `/config`). Se sono presenti entrambi, prevale `CLAUDE.md` e `AGENTS.md` viene ignorato nelle sessioni normali; `/init` lo legge comunque durante la scaffolding. Se il repository usa già `AGENTS.md` per altri agenti di programmazione e servono anche regole specifiche per Claude, mantieni un'unica fonte di verità importandolo:

```
# CLAUDE.md
@AGENTS.md

## Override specifici di Claude
Usa la modalità plan per le modifiche in `src/billing/`.
```

Sintassi @import

I file CLAUDE.md possono importare altri file per una configurazione modulare:

```
# Configurazione del progetto

@docs/coding-standards.md
@docs/api-conventions.md
@~/.claude/my-personal-preferences.md

Consulta @README.md per una panoramica del progetto.
```

Regole di risoluzione:

- I percorsi relativi vengono risolti rispetto al file che contiene l'importazione.
- `@~/` indica la directory home dell'utente.
- Profondità massima di importazione: 5 passaggi (per evitare importazioni circolari).
- Anche i file importati possono contenere @import, fino al limite di profondità.

Directory .claude/rules/

Per organizzare in file separati le regole specifiche per argomento:

```
.claude/
  rules/
    testing.md       — Convenzioni di test
    api-design.md    — Schemi per gli endpoint API
    security.md      — Requisiti di sicurezza
    typescript.md    — Regole specifiche per TypeScript
```

Senza frontmatter: le regole vengono caricate sempre all'avvio della sessione. **Con frontmatter:** le regole vengono caricate in base ai percorsi dei file.

Memoria automatica

Claude Code conserva la memoria automatica in:

```
~/.claude/projects/<project>/memory/MEMORY.md
```

- All'avvio della sessione vengono caricate **le prime 200 righe o 25 KB**, a seconda di quale limite venga raggiunto per primo.
- È locale al computer e non è soggetta al controllo versione.
- Tutti i worktree dello stesso repository Git condividono una directory di memoria.
- Claude vi annota automaticamente schemi del progetto, decisioni e preferenze.
- Richiede Claude Code v2.1.59 o successive.
- `autoMemoryEnabled: false` la disattiva; `autoMemoryDirectory` ne cambia il percorso.
- I file tematici (ad esempio `debugging.md`, `api-conventions.md`) accanto a `MEMORY.md` vengono caricati su richiesta: il limite di 200 righe si applica solo a `MEMORY.md`.

- Struttura attuale: `MEMORY.md` è un **indice** (una riga per memoria) e ogni memoria ha un file dedicato con frontmatter YAML — `name`, `description` e un `type` tra `user` / `feedback` / `project` / `reference`; Claude Code aggiunge un timestamp `modified` a ogni scrittura (dalla versione v2.1.214).
- I subagenti possono mantenere una propria memoria automatica.

Controlli per monorepo e directory multiple

Impostazione / flag	Scopo
`claudeMdExcludes` (nelle impostazioni)	Elenco glob di file CLAUDE.md da ignorare; utile quando altrimenti verrebbe caricato un CLAUDE.md annidato appartenente a un altro team
`--add-dir <path>`	Concede a Claude l'accesso ad altre directory di lavoro
`CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1`	Carica anche CLAUDE.md e le regole dai percorsi specificati con `--add-dir`
`CLAUDE_CODE_NEW_INIT=1`	`/init` interattivo in più fasi che crea CLAUDE.md, skill e hook

3.2 Skill (e comandi slash legacy)

> **Modifica importante (2026):** i comandi slash personalizzati sono stati **integrati nelle skill**. Un file `.claude/commands/deploy.md` e una skill `.claude/skills/deploy/SKILL.md` creano entrambi `/deploy` e funzionano allo stesso modo. Gli spazi dei nomi `/project:name` e `/user:name` non esistono più: i comandi sono semplicemente `/name` e, in caso di nomi duplicati, prevale la posizione con priorità maggiore.

Percorsi delle skill (priorità: aziendale > personale > progetto > plugin)

Posizione	Percorso	Ambito
Aziendale	Percorso delle impostazioni gestite	Tutti gli utenti dell'organizzazione
Personale	`~/.claude/skills/<name>/SKILL.md`	Tutti i tuoi progetti
Progetto	`.claude/skills/<name>/SKILL.md`	Solo questo progetto
Plugin	`<plugin>/skills/<name>/SKILL.md`	Dove il plugin è abilitato (spazio dei nomi `plugin-name:skill-name`)

I file legacy `.claude/commands/*.md` funzionano ancora e supportano lo stesso frontmatter, ma le skill aggiungono una directory per i file di supporto, l'iniezione dinamica del contesto e l'esecuzione dei subagenti. Se una skill e un comando hanno lo stesso nome, prevale la skill. Le skill di Claude Code seguono lo standard aperto Agent Skills.

Esempio di SKILL.md

```
---
name: deploy
description: Distribuisci l'applicazione nell'ambiente specificato
context: fork
agent: Explore
allowed-tools: Bash(git *) Bash(npm run deploy *)
argument-hint: [environment]
disable-model-invocation: true
---

# Skill di distribuzione

Distribuisci l'applicazione in $ARGUMENTS.

1. Esegui la suite di test.
2. Compila l'applicazione.
```

```
3. Pubblica sull'ambiente di destinazione.
4. Verifica che la distribuzione sia riuscita.
```

Riferimento frontmatter (attuale)

Campo	Scopo
`name`	Nome del comando `/slash` (predefinito: nome della directory; minuscole, cifre, trattini, massimo 64 caratteri)
`description`	Quando usare la skill: Claude la usa per decidere se caricarla automaticamente. Massimo 1.536 caratteri insieme a `when_to_use`
`when_to_use`	Ulteriori frasi di attivazione o richieste di esempio, aggiunte a `description`
`argument-hint`	Suggerimento mostrato nel completamento automatico, ad esempio `[issue-number]`
`arguments`	Argomenti posizionali con nome per la sostituzione di `$name`
`disable-model-invocation`	`true` → solo l'utente può invocare la skill tramite `/name`; Claude non può attivarla automaticamente
`user-invocable`	`false` → la skill è nascosta dal menu `/`; Claude può comunque invocarla (conoscenza di contesto)
`allowed-tools`	Strumenti approvati in anticipo mentre la skill è attiva
`model`	Sostituisce il modello per il turno della skill
`effort`	`low` / `medium` / `high` / `xhigh` / `max` — sostituisce il livello di effort
`context`	`fork` esegue la skill in un subagente isolato
`agent`	Tipo di subagente che esegue la skill quando `context: fork` (ad esempio `Explore`, `Plan`, `general-purpose`)
`hooks`	Hook associati al ciclo di vita della skill
`paths`	Pattern glob: carica automaticamente la skill solo quando si lavora su file corrispondenti
`shell`	`bash` (predefinito) oppure `powershell` per l'iniezione di comandi inline

Variabili di sostituzione

Variabile	Viene risolta in
`$ARGUMENTS`	Intera stringa di argomenti passata dall'utente
`$ARGUMENTS[N]` / `$N`	Argomento posizionale con indice da 0 (`$0`, `$1`, ...)
`$name`	Argomento con nome dichiarato nel frontmatter `arguments:`
`${CLAUDE_SESSION_ID}`	Identificatore della sessione corrente
`${CLAUDE_SKILL_DIR}`	Directory che contiene il file SKILL.md

Iniezione dinamica del contesto

Le skill possono elaborare l'output della shell e il contenuto dei file prima che Claude li veda:

```
---
name: pr-summary
description: Riassumi le modifiche di una pull request
context: fork
agent: Explore
allowed-tools: Bash(gh *)
```

```
---

## Contesto della PR
- Diff: !`gh pr diff`
- Commenti: !`gh pr view --comments`
- File modificati: !`gh pr diff --name-only`

## Attività
Riassumi questa pull request...
```

!`cmd` esegue il comando *prima* dell'invio del corpo della skill. Per i blocchi su più righe si usa ```!. È possibile includere anche file con @path. Per disattivare globalmente la funzionalità, imposta "disableSkillShellExecution": true.

Isolamento del contesto (context: fork)

- Avvia la skill come subagente con un proprio contesto.
- Il campo agent: seleziona l'ambiente di esecuzione (subagente integrato Explore, Plan, general-purpose oppure uno personalizzato in .claude/agents/*).
- CLAUDE.md viene comunque caricato; la cronologia della conversazione principale no.
- È utile per operazioni che generano molto contenuto (ricerche approfondite, distribuzioni) e che appesantirebbero il contesto principale.

Ciclo di vita e compattazione delle skill

Quando viene invocata, la versione renderizzata del corpo della skill entra nella conversazione come singolo messaggio e vi rimane. Claude Code **non** rilegge il file nei turni successivi: scrivi istruzioni permanenti, non passaggi da eseguire una sola volta. La compattazione automatica riaggancia le skill invocate più di recente (i primi 5.000 token di ciascuna, entro un budget complessivo di 25.000 token) ed esclude quelle meno recenti se ne sono state usate molte.

Skill incluse

Claude Code include skill predefinite che variano a seconda della versione. Le versioni recenti comprendono:

- /claude-api — Crea e migra applicazioni basate su Claude API / Anthropic SDK; sottocomandi migrate, prompt-audit, upgrade (Python SDK da 0.x a 1.x), cost-optimize, preserved-thinking-migration (individua e corregge le modifiche alla cronologia introdotte dall'harness), build-eval / hillclimb, managed-agents-onboard.
- /code-review — Revisione multiagente delle modifiche in sospeso o di una PR (ultra la esegue nel cloud; /ultrareview è l'alias legacy).
- /diff — Esamina le modifiche nel working tree, comprese quelle di Claude, prima della pubblicazione.
- /security-review — Revisione di sicurezza delle modifiche in sospeso nel branch.
- /simplify — Migliora riuso, semplicità ed efficienza del codice modificato.
- /dataviz — Crea grafici e dashboard con un design coerente.
- /design — Area di lavoro per la progettazione visiva con più tavole.
- /init — Crea lo scheletro di un CLAUDE.md per la codebase.
- /run — Avvia l'applicazione del progetto per verificare che una modifica funzioni.
- /loop — Esegue periodicamente un prompt o un comando slash.
- /schedule — Crea e gestisce routine cloud.
- /deep-research — *Workflow* dinamico incluso: avvia ricerche Web in parallelo, confronta le fonti e sintetizza un rapporto con citazioni.

- `/workflows` / `/workflow-authoring` — Monitora i workflow in esecuzione e carica il riferimento per scrivere script (se i workflow dinamici sono abilitati).
- `/advisor` — Consulta un secondo modello (`fable`, `opus`, `sonnet`) nei momenti chiave di un'attività.
- `/fast` — Attiva o disattiva la modalità rapida (Opus 5.5 / Opus 5 / Opus 4.8; tariffa premium: $8 / $40 per Opus 5.5, $10 / $50 per Opus 5 e 4.8; stesso modello, risposta più rapida; disponibile anche nelle sessioni cloud e remote self-hosted).
- `/update-config` — Configura settings.json, hook e autorizzazioni.
- `/fewer-permission-prompts` — Crea una allowlist di autorizzazioni in base all'utilizzo.
- `/output-style [name]` — Elenca e cambia gli stili di output, anche nelle sessioni senza interfaccia e Remote Control (dalla versione v2.1.269).
- `/insights` — Rapporto sull'utilizzo; ora stima quante richieste di autorizzazione recenti avrebbero potuto essere gestite dalla modalità auto.
- `/skill-doctor` — Mostra il costo in token nel contesto di ciascuna skill e la frequenza d'uso (dalla versione v2.1.252); ogni skill elencata consuma token a ogni turno.
- `/doctor` — Diagnostica i problemi di configurazione di Claude Code; `/feedback` invia un rapporto con il contesto della sessione.

Controlla `/help` nella versione installata (o la pagina dei comandi nella documentazione) per consultare l'elenco aggiornato: cambia da una versione all'altra.

Plugin

Un **plugin** è un pacchetto distribuibile di skill, subagenti, hook e server MCP, installato da un marketplace e abilitato per progetto. Gli elementi forniti dai plugin hanno spazi dei nomi dedicati (`plugin-name:skill-name` per le skill, `plugin:<plugin>:<server>` per i server MCP), così non entrano in conflitto con quelli personalizzati. I plugin hanno la priorità più bassa nell'ordine di risoluzione delle skill (aziendale > personale > progetto > plugin).

I plugin sono anche l'unità verificata da `claude plugin eval`: scrivi una suite di valutazione per le skill del plugin e la esegui in un ambiente isolato o in CI; il comando genera un rapporto JSON. È il modo per distribuire e verificare con test di regressione le convenzioni Claude Code di un team.

Strumenti recenti per i plugin: `claude plugin validate` controlla anche le voci `.mcp.json` del plugin (server che verrebbero ignorati senza avviso, riferimenti `${user_config.*}` non dichiarati, URL non sicuri); `--plugin-dir` può indicare una directory di plugin e rileva i nuovi elementi aggiunti durante l'esecuzione; `claude plugin install <plugin> --marketplace <source>` propone di aggiungere il marketplace durante l'installazione; inoltre, skill e plugin abilitati nell'account claude.ai vengono sincronizzati con le sessioni terminale che usano lo stesso account (imposta `syncClaudeAiSkills` / `syncClaudeAiPlugins: false` per disattivare la sincronizzazione).

3.3 Regole specifiche per percorso

Caricamento condizionale delle regole

Le regole in `.claude/rules/` possono specificare a quali file si applicano usando pattern glob nel frontmatter YAML:

```
---
paths:
  - "src/api/**/*.ts"
  - "src/api/**/*.test.ts"
---

# Regole per lo sviluppo delle API

Tutti gli endpoint API devono:
```

```
- usare Zod per convalidare le richieste;
- restituire risposte di errore standardizzate;
- includere middleware per limitare la frequenza delle richieste;
- avere i relativi test di integrazione.
```

```
---
paths:
  - "**/*.test.tsx"
  - "**/*.test.ts"
  - "**/*.spec.ts"
---

# Convenzioni di test

- Usa React Testing Library, non Enzyme.
- Verifica il comportamento, non l'implementazione.
- Simula le API esterne, non i moduli interni.
- Ogni file di test deve contenere un blocco describe il cui nome corrisponde al componente o alla
funzione.
```

Quando le regole specifiche per percorso sono preferibili a CLAUDE.md nelle directory

Scenario: convenzioni di test applicabili a file distribuiti in tutta la codebase.

Approccio con CLAUDE.md nelle directory:

```
src/
  components/
    CLAUDE.md  ← Qui servirebbero regole di test
    Button.test.tsx
  api/
    CLAUDE.md  ← E anche qui
    users.test.ts
  utils/
    CLAUDE.md  ← E anche qui
    format.test.ts
```

Approccio con regole specifiche per percorso:

```
.claude/
  rules/
    testing.md  ← Un solo file con paths: ["**/*.test.*"]
```

Questo approccio è preferibile perché:

- mantiene un'unica fonte di verità per le convenzioni di test;
- si applica ovunque si trovino file di test, anche in nuove directory;
- evita di duplicare file CLAUDE.md in directory diverse.

3.4 Modalità plan ed esecuzione diretta

Modalità di autorizzazione (attuali)

La modalità plan è una delle opzioni disponibili in un sistema più ampio di modalità di autorizzazione. Avvia una modalità specifica con `--permission-mode <mode>` oppure scorri le modalità con **Shift+Tab** in una sessione interattiva.

Modalità	Comportamento
`default`	Richieste di autorizzazione standard
`acceptEdits`	Accetta automaticamente le modifiche ai file; continua a chiedere conferma per gli altri strumenti
`plan`	Solo esplorazione: Claude progetta un approccio e lo presenta prima di modificare i file
`auto`	Un classificatore decide per ogni strumento se chiedere l'autorizzazione; vedi `claude auto-mode defaults`. Dalle versioni v2.1.278–282, il classificatore viene eseguito **lato server per impostazione predefinita** su Claude API, Bedrock, Vertex, Foundry e gateway (senza addebito dei costi del classificatore; `CLAUDE_CODE_AUTO_MODE_SERVER=0` lo disattiva; `/status` mostra quale modalità è attiva). Anche i comandi in sola lettura o in ambiente isolato attendono la sua verifica.
`dontAsk`	Non chiedere mai conferma per l'elenco preapprovato
`bypassPermissions`	Salta tutte le richieste (pericoloso; usalo solo per attività brevi e circoscritte)

Quando usare la modalità plan

Indicatore	Esempio
Modifiche a più file	Refactoring dell'autenticazione in 15 file
Decisioni architetturali	Scegliere tra REST e GraphQL
Più approcci validi	Diversi modi per implementare la cache
Ambito ampio	Migrazione di una libreria che interessa oltre 45 file
Requisiti poco chiari	«Rendi l'app più veloce»: prima serve un'indagine

In modalità plan, Claude:

1. Esplora la codebase (Glob, Grep, Read).
2. Individua i file e gli schemi coinvolti.
3. Progetta un approccio.
4. Presenta il piano per l'approvazione.
5. Implementa solo dopo l'approvazione dell'utente.

Quando usare l'esecuzione diretta

Indicatore	Esempio
Correzione in un singolo file	Correzione di un bug con stack trace chiaro
Ambito ovvio	Aggiungere un controllo `null`
Istruzioni chiare	«Aggiungi un campo `createdAt` al modello User»
Modifica piccola e circoscritta	Correggere un refuso, aggiornare una costante

Subagenti integrati

Claude Code include quattro subagenti, registrati per impostazione predefinita nelle sessioni interattive:

Agente	Scopo	Strumenti
Explore	Individuazione rapida di file e ricerca nella codebase	Solo lettura; ignora CLAUDE.md e lo stato Git

Agente	Scopo	Strumenti
Plan	Ricerca per la modalità plan prima di proporre modifiche	Solo lettura; ignora CLAUDE.md e lo stato Git
general-purpose	Attività articolate in più passaggi che richiedono sia esplorazione *sia* azione	Tutti gli strumenti per subagenti
claude	Agente di riserva generico	Tutti gli strumenti per subagenti

L'agente Explore è il metodo standard per preservare il contesto:

```
Il contesto della conversazione principale viene preservato
     ↓
Il subagente Explore lavora (Glob, Grep, Read — molto output)
     ↓
Restituisce un riepilogo conciso alla conversazione principale
     ↓
Il contesto principale resta pulito
```

Senza questo approccio, un'esplorazione approfondita della codebase riempirebbe la finestra di contesto principale con contenuti grezzi dei file e risultati di ricerca.

Subagenti personalizzati (`.claude/agents/`)

I subagenti personalizzati sono file Markdown con frontmatter YAML, risolti nel seguente ordine: impostazioni gestite > JSON CLI `--agents` > progetto (`.claude/agents/`) > utente (`~/.claude/agents/`) > plugin.

```yaml
---
name: db-reader
description: Esegui query di sola lettura sul database # come Claude decide di delegare
tools: Read, Grep, Bash                                # elenco consentito
disallowedTools: Write, Edit                           # elenco negato, applicato dopo `tools`
model: sonnet                                          # inherit | sonnet | opus | haiku | ID
completo
effort: low                                            # low | medium | high | xhigh | max
permissionMode: plan                                   # default | acceptEdits | auto | dontAsk |
bypassPermissions | plan
maxTurns: 10
skills: [api-conventions]                              # contenuto delle skill precaricate
mcpServers: [github]
memory: project                                        # user | project | local — persiste tra le
sessioni
isolation: worktree                                    # esegue nel proprio worktree Git
background: false
omitClaudeMd: true                                     # ignora CLAUDE.md utente / progetto /
locale; le policy gestite vengono comunque caricate (dalla v2.1.271)
experimental:
  cacheTtl: 1h                                         # durata della cache del prompt per questo
subagente (dalla v2.1.248)
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: ./scripts/validate-readonly-query.sh
---

Esegui solo query SELECT. Non effettuare mai scritture.
```

Tre modi per invocarlo: nominarlo in linguaggio naturale («usa db-reader per…»), menzionarlo con `@` per garantire l'esecuzione di quello specifico agente oppure avviare un'intera sessione con `claude --agent db-reader`.

Cosa carica un nuovo subagente: il proprio prompt di sistema, il prompt di delega, la gerarchia CLAUDE.md (a meno che `omitClaudeMd: true`), uno snapshot dello stato Git, le skill precaricate e i dettagli dell'ambiente. **Cosa resta escluso:** la cronologia della conversazione, la memoria automatica della sessione principale e i risultati degli altri subagenti: è proprio il principio per cui «il contesto non viene ereditato» (Dominio 1 §1.3).

Vincoli da ricordare: le modalità di autorizzazione si propagano *verso il basso*: se il genitore usa `bypassPermissions`, `acceptEdits` o `auto`, il subagente eredita la modalità e non può cambiarla. Per impostazione predefinita, i subagenti possono annidarsi fino a 3 livelli e possono esserne eseguiti 20 contemporaneamente. I subagenti completati possono essere ripresi per ID o nome tramite `SendMessage`, mantenendo il contesto precedente; gli agenti monouso (Explore, Plan) non possono essere ripresi.

Nota sulla prompt injection: dalla versione v2.1.277, il risultato di un subagente arriva all'agente principale sotto un'intestazione che lo identifica come output del subagente e con il testo rientrato, così il contenuto non può essere scambiato per istruzioni della sessione (nelle versioni precedenti i rapporti venivano analizzati per trovare sequenze simili a istruzioni e neutralizzarle). È una mitigazione, non un confine di autorizzazione: ogni chiamata a uno strumento continua a passare dai controlli di autorizzazione della sessione.

Oltre i subagenti: team di agenti e workflow dinamici

I subagenti sono singoli esecutori attivati uno per delega e scelti turno per turno. Quando non bastano, sono disponibili due unità più ampie. La domanda tipica d'esame è: *chi detiene il piano?*

	Subagenti	**Team di agenti (sperimentale)**	**Workflow dinamici**
Cosa sono	Un esecutore avviato da Claude	Una sessione principale che supervisiona sessioni Claude Code tra pari, con elenco di attività condiviso e messaggistica tra agenti	Uno script JavaScript scritto da Claude ed eseguito in background dal runtime
Chi decide cosa eseguire dopo	Claude, turno per turno	L'agente principale	Lo script (`agent()`, `parallel()`, `pipeline()`, `phase()`)
Dimensioni	Alcune attività delegate per turno	Pochi agenti tra pari attivi a lungo	Da decine a centinaia di agenti per esecuzione; riprendibile nella sessione
Attivazione	Integrati	`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`	Piani a pagamento e accesso API; nei piani Pro si attiva in `/config`; `workflowSizeGuideline` imposta il numero consultivo di agenti (per un workflow medio il valore è ora 10, nei piani Pro il predefinito è small); `CLAUDE_CODE_WORKFLOW_MAX_CONCURRENT_AGENTS` (1–256) aumenta il limite di concorrenza per esecuzione

Un workflow viene eseguito solo se l'utente lo richiede: selezionando il livello di effort `ultracode`, chiedendo esplicitamente «usa un workflow» oppure invocando una skill inclusa come `/deep-research`. `/workflows` consente di monitorare, sospendere, riprendere o salvare un'esecuzione. L'obiettivo è codificare l'orchestrazione: il piano diventa uno script verificabile e rieseguibile, anziché dipendere dalla memoria del coordinatore (Dominio 1 §1.2).

Combinare le modalità

Modalità plan per l'indagine → esecuzione diretta per l'implementazione:

```
1. Entra in modalità plan.
2. Esplora la codebase.
3. Progetta l'approccio.
4. Presenta il piano all'utente.
5. Attendi l'approvazione dell'utente.
6. Esci dalla modalità plan.
7. Implementa direttamente, un file alla volta.
```

3.5 Tecniche di perfezionamento iterativo

1. Esempi concreti

Quando le istruzioni in prosa producono risultati incoerenti, fornisci 2-3 esempi di input e output:

```
Invece di: "Formatta le date in modo coerente"

Usa: "Formatta le date come AAAA-MM-GG. Esempi:
  Input: '5 gennaio 2024' → Output: '2024-01-05'
  Input: '31/12/23' → Output: '2023-12-31'
  Input: 'mar 2024' → Output: '2024-03-01'"
```

2. Iterazione TDD

Scrivi prima i test, poi procedi per iterazioni condividendo gli errori:

```
1. Scrivi il test: expect(validateEmail("bad")).toBe(false)
2. Esegui i test → NON RIUSCITI
3. Condividi l'errore con Claude.
4. Claude corregge l'implementazione.
5. Esegui i test → SUPERATI oppure nuovi errori.
6. Ripeti finché tutti i test sono superati.
```

3. Schema a intervista

Claude pone domande di chiarimento prima di implementare:

```
Utente: "Aggiungi l'autenticazione all'app"
Claude: "Prima di implementarla, ho alcune domande:
  - JWT o basata su sessione?
  - Servono token di aggiornamento?
  - Quanto deve durare la sessione?
  - Devo aggiungere il controllo degli accessi basato sui ruoli?"
```

4. Problemi interdipendenti e indipendenti

Più problemi interdipendenti: affrontali in un unico messaggio, perché le correzioni potrebbero entrare in conflitto.

```
"Il middleware di autenticazione ha due problemi:
1. Non controlla la scadenza del token.
2. Restituisce 500 invece di 401 per i token non validi.
Correggili entrambi insieme."
```

Problemi indipendenti: correggili in sequenza, così ogni modifica resta chiara e facile da revisionare.

3.6 Integrazione CI/CD

Flag CLI per CI

Flag	Scopo	Critico?
`-p` / `--print`	Modalità non interattiva	**Obbligatorio in CI** per evitare che il processo resti in attesa di input
`--output-format`	`text`, `json` o `stream-json` per un output analizzabile da una macchina	Necessario per l'elaborazione successiva
`--input-format stream-json`	Trasmette gli eventi di input all'agente	Per comporre pipeline
`--json-schema '{...}'`	Convalida l'output finale rispetto a uno schema JSON (solo modalità print)	Impone la struttura dell'output CI
`--max-turns N`	Limite rigido ai turni agentici; se raggiunto, termina con un errore	Protezione
`--max-budget-usd X`	Si ferma quando la spesa API raggiunge X $ (solo modalità print)	Limite di spesa
`--bare`	Salta la rilevazione automatica (hook, skill, plugin, MCP, memoria automatica, CLAUDE.md) per avvii rapidi tramite script	Chiamate script con overhead ridotto
`--no-session-persistence`	Non salva la sessione su disco	Esecuzioni CI effimere
`--fallback-model sonnet`	Passa automaticamente a un modello di riserva se quello principale è sovraccarico	Affidabilità
`--include-hook-events`	Emette nel flusso tutti gli eventi del ciclo di vita degli hook	Osservabilità
`--include-partial-messages`	Emette gli eventi parziali dello streaming	Debug
`--permission-mode <mode>`	Avvia in modalità `default` / `acceptEdits` / `plan` / `auto` / `dontAsk` / `bypassPermissions`	Vedi §3.4
`--permission-prompt-tool <mcp>`	Delega le richieste di autorizzazione a uno strumento MCP in modalità senza interfaccia	Approvazioni non interattive
`--permission-prompts none`	Nega le operazioni che richiederebbero conferma quando nessuno può rispondere (dalla v2.1.259); la modalità attiva continua a determinare tutto il resto	Runner non presidiati
`--restricted`	Rimuove gli strumenti per eseguire comandi/codice e `WebFetch`, limita gli strumenti sui file alle directory di lavoro, carica solo impostazioni gestite e rifiuta `bypassPermissions` (dalla v2.1.248)	Harness di valutazione su computer condivisi
`--safe-mode`	Avvia disattivando tutte le personalizzazioni (CLAUDE.md, skill, hook, MCP, plugin, workflow, memoria automatica); a differenza di `--bare`, mantiene attive le autorizzazioni	Diagnosi di una configurazione non funzionante
`--session-id`, `--resume`, `--continue`, `--fork-session`	Gestisce il ciclo di vita della sessione	Pipeline articolate in più passaggi
`--setting-sources user,project,local`	Limita i livelli di impostazioni caricati	Esecuzioni riproducibili

Flag	Scopo	Critico?
`--effort <level>`	`low` / `medium` / `high` / `xhigh` / `max` / `ultracode`	Regola il rapporto tra costo e qualità per ogni esecuzione
`--agent <name>` / `--agents '<json>'`	Esegue la sessione come subagente con nome oppure definisce subagenti inline	Attività CI specializzate
`--append-subagent-system-prompt` / `-file`	Aggiunge testo (o un file, dalla v2.1.261) al prompt di ogni subagente, tranne i fork (solo modalità non interattiva)	Regole coerenti per citazioni e formattazione
`--forward-subagent-text`	Emette nel flusso i messaggi dei subagenti	Osservabilità del lavoro delegato
`--system-prompt` / `--system-prompt-file` / `--append-system-prompt`	Sostituisce o estende il prompt di sistema (per prompt lunghi usa le varianti `-file`: i runner self-hosted ora passano i prompt come file; `--system-prompt-snapshot off` lo rigenera a ogni richiesta mentre modifichi il testo)	Profili CI dedicati
`--exclude-dynamic-system-prompt-sections`	Esclude dal prompt di sistema le sezioni specifiche del computer	Riutilizzo della cache del prompt tra runner
`--strict-mcp-config`	Usa solo i server indicati con `--mcp-config`	Ambiente MCP isolato
`--init` / `--init-only`	Esegue gli hook Setup / SessionStart e termina	Prepara un workspace CI prima dell'esecuzione effettiva
`--bg` / `--background` + `--exec`	Avvia un agente in background; esegue un comando shell come attività in background	Attività lunghe che non devono bloccare
`--environment ccpool_*` / `--ref`	Seleziona un ambiente self-hosted e una ref specifica	Esecuzioni distribuite o remote
`--betas`	Invia intestazioni beta con le richieste API	Attiva funzionalità in anteprima

Esempio di utilizzo in CI

```
# Esempio GitHub Actions
- name: Revisione del codice
  run: |
    claude -p "Esamina questa PR per individuare bug e problemi di sicurezza" \
      --output-format json \
      --json-schema '{"type":"object","properties":{"issues":{"type":"array"}}}'
```

Integrazioni CI gestite

Non è sempre necessario configurare la CLI manualmente:

Integrazione	Funzione
GitHub Actions	Esegue Claude Code in un workflow: revisione PR, classificazione delle issue, menzioni `@claude`
GitLab CI/CD	Stesso modello nelle pipeline GitLab
GitHub Code Review	Revisione automatica di ogni PR, senza dover mantenere un file workflow
Routines	Esecuzioni pianificate nel cloud, attivabili anche da eventi GitHub o chiamate API

La combinazione `-p` + `--output-format json` resta la soluzione di base quando serve il pieno controllo; queste integrazioni offrono invece un percorso gestito.

Controlli di costo, cache e flotta (impostazioni)

Alcune impostazioni utili negli scenari in cui un team di piattaforma deve mantenere economiche e coerenti le sessioni di 200 sviluppatori:

Impostazione	Scopo
`promptCacheTtl` / `subagentPromptCacheTtl`	Sceglie una durata della cache del prompt di 5 minuti o 1 ora per la sessione e i subagenti (override per agente: `experimental.cacheTtl` nel frontmatter)
`modelPicker` / `modelPricing` (gestite)	Personalizza l'elenco `/model`; imposta tariffe personalizzate per token affinché `/cost` sia preciso dietro un gateway (un `multiplier` fino a 10 applica un ricarico alle tariffe interne)
`managedMcpServers` (gestita)	Server MCP forniti dall'organizzazione e disponibili in ogni sessione
`bashOutputMaxChars`	Limita l'output Bash inline (fino a 128K caratteri) prima che saturi il contesto. `taskOutputMaxChars` non ha più effetto: lo strumento TaskOutput è stato rimosso nella v2.1.277 e Claude legge con Read il file di output dell'attività in background
`permissions.blockReadsOutsideWorkingDirectories`	Limita la lettura dei file alle directory di lavoro
`workflowSizeGuideline`	Limite consultivo al numero di agenti nei workflow dinamici
`effortLevel` / `maxEffortLevel`	Effort ora è **specifico per modello**: un livello salvato per un modello non viene trasferito a un modello rilasciato in seguito (Opus 5.5 parte dal proprio valore predefinito finché non ne scegli uno con `/effort`); `maxEffortLevel` (al livello principale o per modello in `modelSettings`) ne imposta il limite su ogni provider. Il modello Opus predefinito di Claude Code è 5.5 e i piani Pro / Team Standard ora usano Opus anziché Sonnet come valore predefinito
`"attribution": false`	Nasconde tutte le attribuzioni nei commit e nelle PR (dalla v2.1.281; le CLI meno recenti ignorano un file di impostazioni che contiene questo valore, quindi nei file condivisi tra versioni mantieni la forma a oggetto)
`syncClaudeAiSkills` / `syncClaudeAiPlugins`	Disattiva la sincronizzazione di skill e plugin abilitati su claude.ai nelle sessioni terminale

Ora `/cost` riporta statistiche della cache del prompt e diagnostica dei cache miss per sessione: è la prospettiva di Claude Code sul problema della stabilità del prefisso descritto nel Dominio 4 §4.8.

API Message Batches

Per operazioni batch non bloccanti e ottimizzate nei costi:

Funzionalità	Dettaglio
Risparmio sui costi	50% rispetto all'API sincrona
Finestra di elaborazione	Fino a 24 ore, senza SLA sulla latenza

Funzionalità	Dettaglio
Correlazione	`custom_id` per associare le risposte alle richieste: i risultati arrivano in **ordine qualsiasi**, quindi usa `custom_id` come chiave, mai la posizione
Più turni	NON supportati all'interno di una singola richiesta batch
Output esteso	Fino a 300.000 token di output con Opus 5.5 / 5 / 4.8 / 4.7 / 4.6 e Sonnet 5 / 4.6 (non Fable 5.1), con intestazione beta `output-300k-2026-03-24` (rispetto ai 128.000 sincroni)

Adatto per:

- Rapporti di analisi del codice da generare durante la notte.
- Riepiloghi settimanali di audit.
- Generazione notturna di test.
- Elaborazione di documenti in batch.

Non adatto per:

- Controlli prima del merge (bloccanti, richiedono una risposta immediata).
- Interazioni utente in tempo reale.
- Qualsiasi attività che richieda loop di chiamate agli strumenti all'interno di una singola richiesta.

Gestione dei fallimenti nei batch

```
# Elabora i risultati del batch
for result in batch_results:
    if result.status == "failed":
        failed_ids.append(result.custom_id)

# Invia nuovamente solo le richieste non riuscite
resubmit_batch = [r for r in original_requests if r.custom_id in failed_ids]
```

Isolamento del contesto di sessione per le revisioni

Problema: la stessa sessione Claude che ha scritto il codice è meno efficace nel revisionarlo, perché conserva il contesto del ragionamento (sa già «perché» ha preso ogni decisione ed è quindi meno incline a metterla in discussione).

Soluzione: usa sessioni di revisione indipendenti, prive del contesto di generazione.

```
Sessione di generazione: Claude scrive il codice
                         ↓
               (nuova sessione, senza contesto condiviso)
                         ↓
Sessione di revisione: Claude esamina il codice senza preconcetti
```